

Using AnimatedSprite/Sprite:

- Entity stores a Sprite* object. This can store AnimatedSprite or Sprite. If you want to properly copy it, use Clone().
-

Class structure:

Each class has descriptions for what fits it.

Collidable: Anything that can be collided with

Element: Anything that can exist in a scene(Text, Entity, Particle Emitter, etc.)

Entity(Element, Collidable): A thing with an orientation that exists in space

Attackable(Entity): A living entity

AI(Attackable): A living thing that moves automatically

Give it a target with SetTarget

Any enemy class(AI): A unique enemy definition

Defines what target it should pick(behavior)

GameContext:

A struct that holds any single instance objects like the 1 audio manager, the 1 texture manager, or the 1 grid, etc.

Also holds a function pointer to change the scene from anywhere in the code. Each scene is given an index.(-1 is quit game)

Also holds mt19937 object called gen to be used universally for any random generation.

Grid:

The all knowing grid. It is the manager of all gridCells and is accessible from anywhere using context

Updates any entities occupancy. Each entity than can added to the grid needs a gridCell add/remove function to manage gridCells storage of the object. Function pointers to these functions must be passed into UpdateOccupancy() (located in Grid.hpp because reasons)

GridCells:

Holds information about each individual object. Entities know which cells they are in via "occupancy"

Any new thing that a cell will hold needs new managing functions to properly add, remove, access, and possibly position inside the cell.

Updating occupancy is really important: Any entity that does not have this updated cannot be detected for collision or detected by the AI. This has been the source of many hours of debugging for me because I forgot.

Collision:

For any entity in the grid with the occupancy updated, you can call HasCollision(entity) to know if the entity is colliding with anything nearby, or ResolveCollisions(entity) which calls the handleCollision function on itself and object its colliding with for each collision.

Inputs:

Inputs are defined in /data/inputs.json

Use the functions in InputManager (context.im) to check a key binding state or mouse state.